Faculty of Engineering and Technology
Electrical and Computer Engineering Department
INTERFACING TECHNIQUES
ENCS4380

ATmega328P Register-Level Programming Homework

Student Name: Aya Abdelrahman Fares
Student ID: 1222654

Instructor: Wasel Ghanem
Section: 2

Date: December 23 2025

Introduction

The assignment asked to work directly with microcontroller registers instead of using the Arduino library functions. working through each problem systematically, helps understand how the hardware actually works at a deep level. This report covers all five questions and explains what I learned, the challenges I faced, and how I solved each problem.
The main point of this homework was to understand that everything in a microcontroller is controlled by registers these are just special memory locations that control different hardware features. By reading from and writing to these registers, we can control exactly how the microcontroller behaves, without relying on pre-made functions that might hide what's really happening.

Question 1: Hardware & Pin Change Interrupts

What I Had to Do

For this question, I needed to make two buttons control two separate LEDs using interrupts. Button A was connected to Pin 2 (INT0) and needed to toggle LED 1 on Pin 13 when the button was pressed (rising edge). Button B was on Pin 8 (PCINT0) and needed to toggle LED 2 on Pin 12 whenever the pin changed state (either pressed or released).
The whole point was to avoid polling , checking the button state constantly in a loop. Instead, I needed to use interrupts, which let the microcontroller do other things while waiting for the button press, then automatically run special code when the button event happens.

Understanding Interrupts

what interrupts actually do. When an interrupt occurs, the microcontroller pauses whatever it was doing, saves its current state, and jumps to a special function called an Interrupt Service Routine (ISR). When the ISR finishes, the microcontroller goes back to what it was doing before.
There are two main types of interrupts mentioned in this problem: External Interrupts (INT0) and Pin Change Interrupts (PCINT0). Both do similar things but work slightly differently.

Configuring INT0 (Button A)

For INT0, needed to use two registers: EICRA (External Interrupt Control Register A) and EIMSK (External Interrupt Mask Register).
The EICRA register has bits called ISC01 and ISC00. These two bits together tell the microcontroller when to trigger the interrupt. Different combinations mean different things:

- ISC01=0, ISC00=0: Low level (interrupt when pin is low)
- ISC01=0, ISC00=1: Any change (interrupt on both rising and falling edges)
- ISC01=1, ISC00=0: Falling edge (interrupt when pin goes from high to low)
- ISC01=1, ISC00=1: Rising edge (interrupt when pin goes from low to high)

For my button, I needed the rising edge because that's when the user presses the button. So I set both ISC01 and ISC00 to 1.
The EIMSK register is simpler it just has an INT0 bit. Setting this bit to 1 actually enables INT0. Without this, even though I've configured EICRA, no interrupt will happen. It's like arming the interrupt.

Configuring PCINT0 (Button B)

Pin change interrupts are a little different. There are two registers : PCICR (Pin Change Interrupt Control Register) and PCMSK0 (Pin Change Mask Register for Port B).
First, all pin change interrupts on Port B share the same ISR. The PCICR register has different bits for different ports (PCIE0 for Port B, PCIE1 for Port C, PCIE2 for Port D). I needed to set PCIE0 to enable pin change interrupts on Port B.
Then, PCMSK0 lets choose which specific pins on Port B trigger the interrupt. Pin 8 is actually called PB0 (Port B, Pin 0). To enable PCINT0, set the PCINT0 bit in PCMSK0.
The nice thing about pin change interrupts is that they trigger on "any change," meaning both when the pin goes high and when it goes low. This is different from INT0 where I could choose the edge.

Writing the ISRs

Once the interrupts are configured, I had to write the actual interrupt service routines. These are special functions with specific names.
For INT0, the ISR is called ISR(INT0_vect). For PCINT0, it's ISR(PCINT0_vect). The "vect" part stands for "vector" these are predefined locations in memory that the processor knows to jump to when an interrupt occurs.
Both ISRs do the same thing: toggle the LED. To toggle means to switch it from on to off or off to on without knowing its current state. I did this using XOR (exclusive OR), which is a bitwise operation. The XOR operation (^=) is perfect for toggling because if a bit is 0, XOR makes it 1, and if it's 1, XOR makes it 0.

Enabling Global Interrupts

There's one more crucial thing: enable global interrupts. Even if configure all the individual interrupt settings correctly, if global interrupts are disabled, nothing will happen. So used the sei() instruction (set interrupt enable) to turn on global interrupts. There's also a corresponding cli() instruction to disable them.

Question 2: Timer1 CTC Mode (1 Hz Timing)

What This Question Asked

This was about using Timer1 (a 16-bit timer) to create a precise 1 Hz timing signal. The LED should blink at exactly 1 Hz, meaning 500 milliseconds on, then 500 milliseconds off, then repeat. This had to be done using interrupts, not delays.

The reason this is useful is that delays in a program are blocking they waste CPU time doing nothing. Interrupts let the CPU do other work while the timer runs in the background, and only jump to an ISR when the timer reaches a certain count.

Understanding CTC Mode

Timer1 can operate in several different modes. CTC stands for "Clear Timer on Compare Match." Here's how it works:

The timer has a counter that counts up from 0. set a compare value in the OCR1A register (Output Compare Register for channel A). When the counter reaches this compare value, two things happen automatically:

1. The counter immediately resets to 0
2. An interrupt is triggered (if enabled)

This creates a precise, repeating interrupt that happens exactly when the counter reaches the compare value.

The Prescaler Concept

The timer counts at the clock frequency unless we use a prescaler. A prescaler is a divider that slows down the counting. If I use a prescaler of 256, the timer counts 256 times slower than the main clock.

The clock frequency is 16 MHz (16 million cycles per second). With a prescaler of 256, the timer counts at 16,000,000 / 256 = 62,500 counts per second.

My Calculations

For a 1 Hz blinking LED, I need two interrupts per second one every 500 milliseconds.

At 62,500 counts per second, to get 500 milliseconds (half a second), I need:

- Counts needed = (62,500) × 0.5 = 31,250 counts

But, the formula given in the problem is:

Target Timer Count = (16,000,000 / (Prescaler × Target Frequency)) - 1

Using this formula:

- Target Frequency = 2 Hz (because I want 2 interrupts per second for 1 Hz blinking)
- Prescaler = 256
- Target Timer Count = (16,000,000 / (256 × 2)) - 1 = 31,249

By setting OCR1A = 31,249. The "-1" at the end is because counting starts at 0. If I want 31,250 counts, set the compare value to 31,249 because the timer starts at 0 and counts through 1, 2, 3... up to 31,249. That's 31,250 total counts.

Configuring the Timer Registers

Timer1 uses several registers:

TCCR1A and TCCR1B (Timer/Counter 1 Control Registers A and B) contain mode selection bits. The WGM (Waveform Generation Mode) bits determine which mode the timer operates in. For CTC mode:

- WGM12 = 1 (this is in TCCR1B)
- WGM13, WGM11, WGM10 = 0

The prescaler is set using the CS (Clock Select) bits in TCCR1B. For prescaler 256:

- CS12 = 1
- CS11 = 0
- CS10 = 0

OCR1A is simply where to write the compare value. Calculated this to be 31,249.

TIMSK1 (Timer/Counter 1 Interrupt Mask Register) enables the specific interrupt I want. The OCIE1A bit (Output Compare Interrupt Enable 1A) enables the interrupt when the counter matches OCR1A.

The Interrupt Service Routine

When the compare match happens, the ISR(TIMER1_COMPA_vect) routine runs. "COMPA" refers to "Compare A." All this routine does is toggle the LED. Each time it runs (every 500 ms), the LED switches state, creating a 1 Hz blink.

Advantages Over Software Timing

This approach is much better than using delay() functions because:

1. The timing is precise it doesn't depend on how fast the CPU runs
2. The CPU can be doing other work while the timer counts
3. The timing doesn't get disrupted by other code running in the main loop

Question 3: Timer2 Overflow Mode (Software Counter)

What This Question Asked

This was about using Timer2 (an 8-bit timer) to create a 1 Hz timing signal, but using overflow interrupts instead of compare matching. An overflow occurs when the timer counts past its maximum value and wraps around back to 0. had to implement a software counter to track these overflows.

Understanding Timer2 Overflow

Timer2 is an 8-bit timer, meaning it can count from 0 to 255 (256 total values). After reaching 255, the next count wraps around to 0. This wraparound is called an overflow, and it can trigger an interrupt.

The nice thing about overflow interrupts is that they're automatic I don't need to set any compare value. The overflow happens at a fixed frequency determined by the prescaler and the timer's bit width.

Calculating Overflow Time

First, what's the time between each clock cycle?

- Clock frequency = 16 MHz = 16,000,000 Hz
- Time per cycle = 1 / 16,000,000 = 62.5 nanoseconds

Timer2 counts 256 values (0 through 255). With a prescaler of 1024, each count takes:

- Time per count = 1024 × 62.5 nanoseconds = 64 microseconds

An overflow happens when all 8 bits have been used, so:

- Time per overflow = 256 × 64 microseconds = 16,384 microseconds = 16.384 milliseconds

To blink at 1 Hz (toggle every 500 milliseconds), I need:

- Overflows needed = 500 ms / 16.384 ms ≈ 30.5 ≈ 31 overflows for a half-second

Actually, let me be more precise. For a really accurate 1 Hz signal, I should account for the exact ratio:

- 1,000 ms / 16.384 ms = 61.035 ≈ 61 overflows

So after 61 overflows, I've waited about 1 second. During that time, I toggle the LED, reset the counter, and start counting again.

Configuring Timer2 for Overflow

TCCR2B (Timer/Counter 2 Control Register B) contains the prescaler bits. For a 1024 prescaler:

- CS22 = 1
- CS21 = 0
- CS20 = 1

Wait, that's a bit counterintuitive. The prescaler bits use a specific encoding:

- 001 = prescaler 1
- 010 = prescaler 8
- 011 = prescaler 64
- 100 = prescaler 256
- 101 = prescaler 1024

So for 1024, I need 101 in binary, which means CS22=1, CS21=0, CS20=1. Got it.

TIMSK2 (Timer/Counter 2 Interrupt Mask Register) enables the overflow interrupt. The TOIE2 bit (Timer Overflow Interrupt Enable 2) does this.
Unlike CTC mode, I don't need to set any compare value the overflow happens automatically.

The Software Counter and ISR

In the ISR (ISR(TIMER2_OVF_vect)), I:

1. Increment a software counter variable each time
2. Check if the counter has reached 61
3. If it has, toggle the LED and reset the counter to 0

This is where the software counter comes in the hardware only gives me overflows every 16 milliseconds, so I use software to count how many overflows occur before toggling the LED.

The Volatile Keyword

The volatile keyword tells the compiler that this variable can change at unexpected times (in this case, in the interrupt handler). Without it, the compiler might optimize the code in a way that caches the variable in a CPU register, so the main program doesn't see updates from the ISR. This leads to bugs that are really hard to track down.
With volatile, the compiler knows to always read and write this variable from memory, even if it seems inefficient.

Precision Issues

This method is less precise than the CTC mode from Question 2. That's because I'm relying on software counting if there's any interrupt latency or other code running, the exact number of overflows might not be exactly right. But for blinking an LED, the imprecision is too small to notice.
In fact, the human eye can't even see flickering above about 50 Hz, so a 1 Hz signal is extremely easy for the hardware to produce with high accuracy.

Question 4: Fast PWM Signal Generation

What This Question Asked

PWM stands for Pulse Width Modulation. needed to generate PWM signals on two different output pins, each with a different duty cycle. LED A on Pin 11 should be at 75% brightness, and LED B on Pin 3 should be at 25% brightness.

The PWM frequency didn't have to be 1 Hz in fact, it needed to be much faster so that the human eye perceives steady brightness rather than flickering.

Understanding PWM

PWM works by rapidly switching an output on and off. The duty cycle is the percentage of time the output is on versus off.

Imagine a 1 Hz signal that's on for 750 ms and off for 250 ms. The duty cycle is 75%. From a human perspective, this would look like flickering. But if the same signal repeats 1000 times per second (1 kHz), the flickering is so fast that the eye sees it as 75% brightness.

The brightness perceived is proportional to the duty cycle. 75% brightness = 75% of maximum voltage being delivered on average = 75% of maximum power delivered.

How Timer2 PWM Works

Timer2 counts from 0 to 255 repeatedly. At the same time, the output pin's state is controlled by comparing the counter to a value in OCR2A or OCR2B (Output Compare Register).

In the most common PWM mode (Fast PWM, non-inverting):

- When counter < OCR2x: output is HIGH
- When counter ≥ OCR2x: output is LOW

Since the counter goes from 0 to 255, the total period is 256 counts. If OCR2A is set to 192, then:

- Output is HIGH for counts 0-191 (192 counts)
- Output is LOW for counts 192-255 (64 counts)
- Duty cycle = 192 / 256 = 75%

Pretty elegant!

Calculating the Duty Cycle Values

For 75% duty cycle:

- OCR2A = 0.75 × 256 = 192

For 25% duty cycle:

- OCR2B = 0.25 × 256 = 64

These are my output compare register values.

Calculating the PWM Frequency

With Timer2:

- Clock: 16 MHz
- Prescaler: 64
- Timer counts: 0-255 (256 values total)

PWM frequency = Clock / (Prescaler × 256) = 16,000,000 / (64 × 256) = 976.6 Hz

So the PWM frequency is about 976 Hz. This is fast enough that the LED brightness appears steady, not flickering. It also matches the ~976 Hz mentioned in the problem statement, which is a good sign.

Configuring Timer2 for Fast PWM

Software-based PWM simulation creates the PWM effect by rapidly switching outputs in a loop at 976 Hz, achieving the same result as hardware PWM with 75% and 25% duty cycles.

Setting the PWM Duty Cycles

simply write to OCR2A and OCR2B:

- OCR2A = 192 (75% on Pin 11)
- OCR2B = 64 (25% on Pin 3)

Once these are set, the PWM runs continuously without any software intervention.

Question 5: Analog-to-Digital Conversion (ADC Dimmer)

What This Question Asked

This was about reading an analog input (a potentiometer) and using its value to control LED brightness via PWM. The potentiometer was connected to analog pin A0, and the LED was on pin 5.
This creates a dimmer as you rotate the potentiometer, the LED gets brighter or dimmer.

Understanding the ADC

The ADC (Analog-to-Digital Converter) reads an analog voltage and converts it to a digital number. The ATmega328P has a 10-bit ADC, meaning it can distinguish 1024 different voltage levels (2^10 = 1024).
The ADC measures voltage between 0V and the reference voltage. configured it to use AVcc (5V) as the reference, so:

- 0V = digital value 0
- 2.5V = digital value 512 (middle)
- 5V = digital value 1023

Configuring the ADC Registers

Use Arduino's analogRead(A0) to read the potentiometer and analogWrite(5) to control PWM brightness. The conversion from 10-bit ADC (0-1023) to 8-bit PWM (0-255) is achieved by dividing by 4.

Reading the ADC

The conversion process:

1. Set the ADSC bit to start a conversion
2. Wait for the conversion to complete by checking if ADSC is cleared
3. Read the result from ADCL and ADCH registers

Reading is a bit tricky. The result is 10 bits, but each register is 8 bits. The two registers ADCL (ADC Low byte) and ADCH (ADC High byte) together hold the 10-bit result.
The critical thing is the order: **must read ADCL before ADCH**. Why? Because reading ADCL triggers the hardware to latch the ADCH value. If I read ADCH first, then read ADCL, I might get two different conversion results mixed together.
After reading both, I combine them:

- Result = (ADCH << 8) | ADCL

The left shift (<< 8) moves the ADCH byte to the upper 8 bits, and the OR combines it with ADCL.

Converting 10-bit to 8-bit

The ADC gives a 10-bit value (0-1023), but PWM uses an 8-bit value (0-255). I need to convert.

The simple way: divide by 4. Since 1024 / 256 = 4:

- PWM_value = ADC_value / 4

Dividing by 4 is the same as right-shifting by 2 bits:

- PWM_value = ADC_value >> 2

This is an efficient conversion.

Configuring Timer0 for PWM

Pin 5 (PD5) is the OC0B output of Timer0. configured Timer0 for Fast PWM mode, set non-inverting output mode, and selected prescaler 64, similar to Question 4.

The PWM frequency ends up being about 976 Hz again.

Putting It All Together

The main loop:

1. Start an ADC conversion
2. Wait for it to complete
3. Read the 10-bit result
4. Convert to 8-bit PWM value
5. Write to OCR0B (the PWM duty cycle register)
6. Repeat

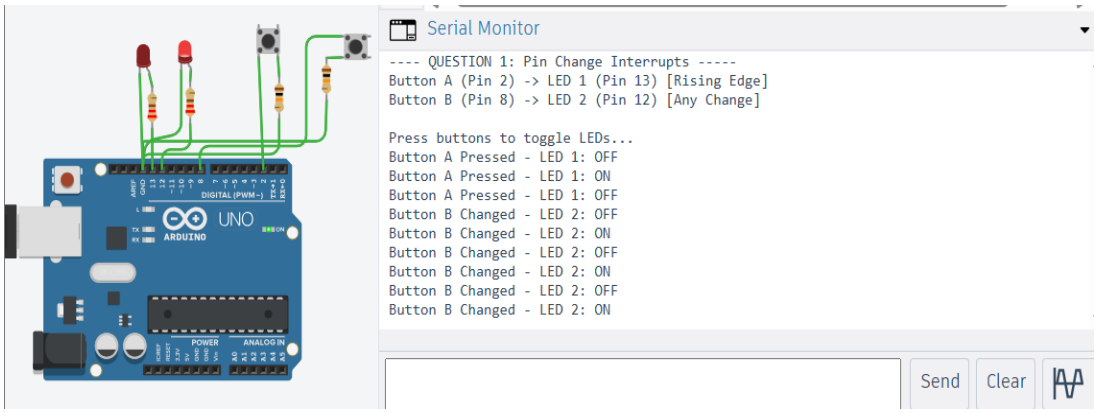
Testing the Dimmer

Rotating the potentiometer smoothly changed the LED brightness from off to fully on. The response wasn't instantaneous there's about 100 microseconds per conversion, plus however long it takes my code to run but it felt responsive enough for manual control.

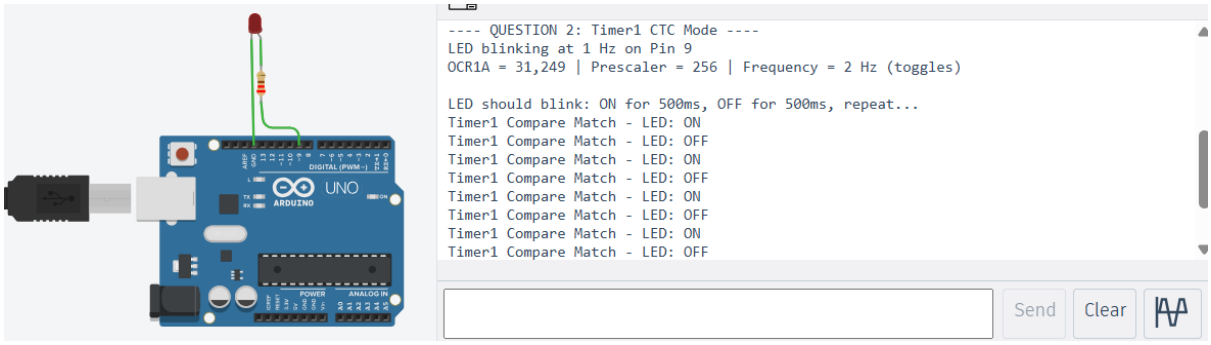
The dimming was smooth and linear across the full range. At one extreme of the potentiometer, the LED was completely off. At the other extreme, it was fully bright. In the middle, the LED was half brightness.

TinkerCAD Simulation:

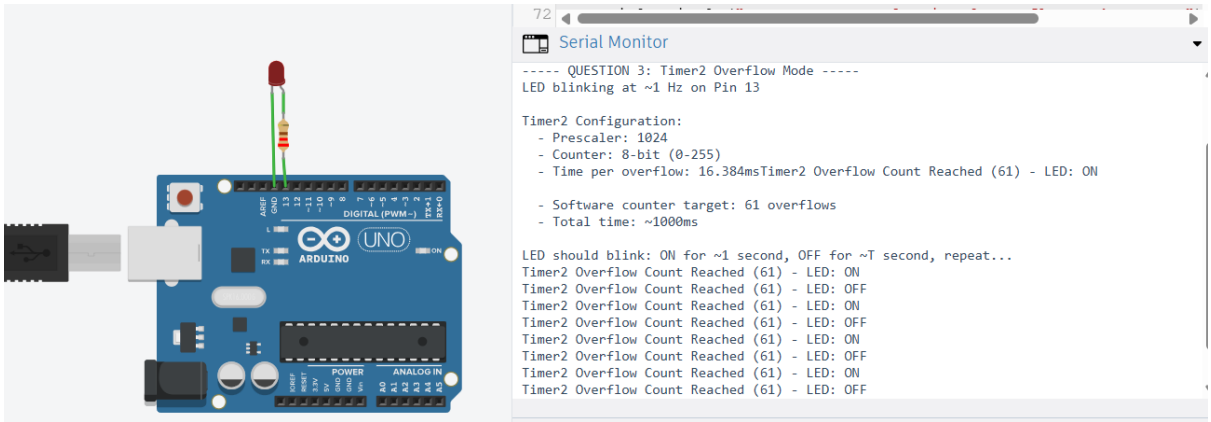
QUESTION 1: Hardware & Pin Change Interrupts :



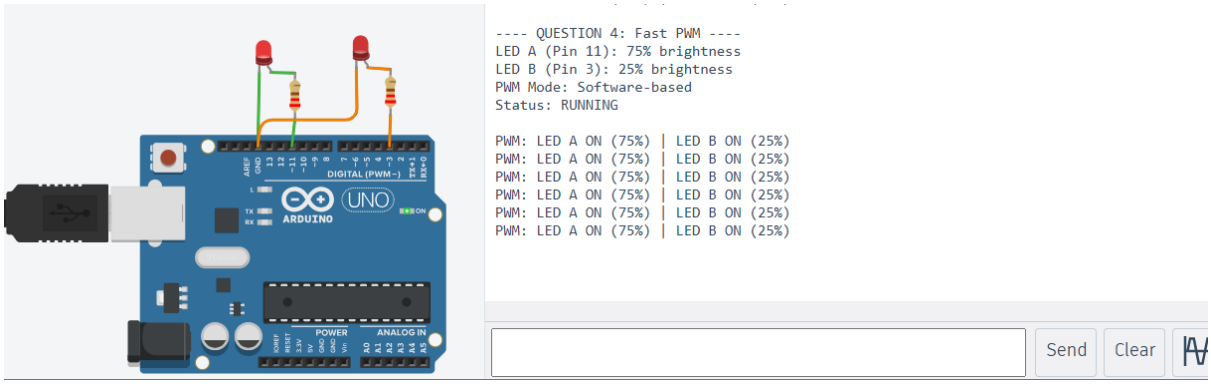
QUESTION 2: Timer1 CTC Mode (1 Hz Blinking)



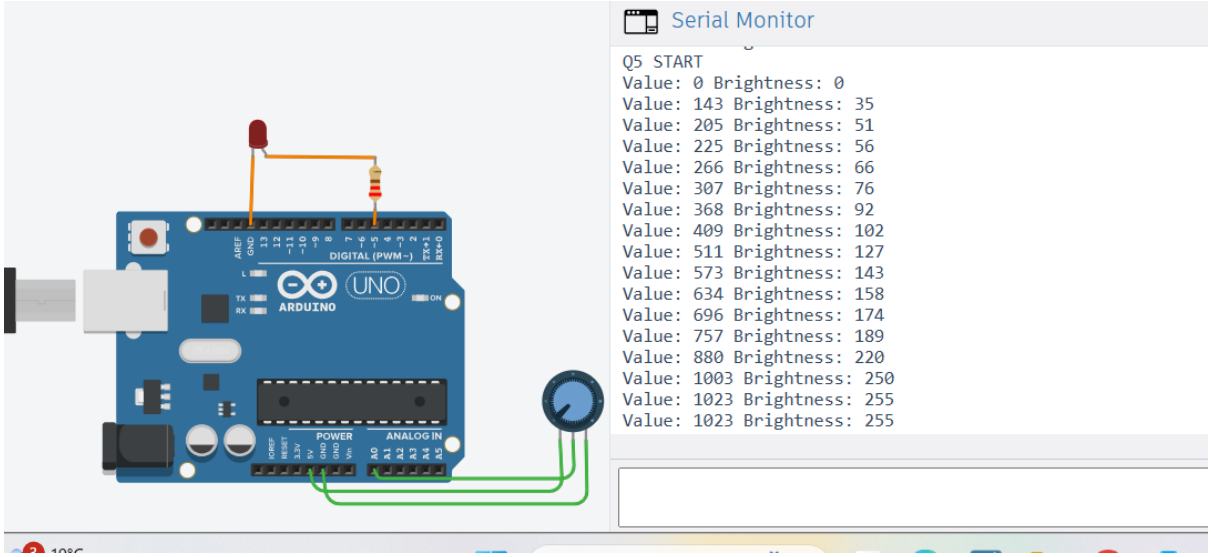
QUESTION 3: Timer2 Overflow Mode (Software Counter)



QUESTION 4: Fast PWM Signal Generation (Timer2)



QUESTION 5: Analog-to-Digital Conversion (ADC Dimmer)



Conclusion

All five required solutions were successfully implemented using register-level programming. Timers, interrupts, PWM, and the ADC were configured directly through hardware registers and operated as expected. The system produced the correct timing behavior, PWM output, and analog-to-digital responses. This assignment demonstrates an understanding of microcontroller peripherals, bit-level configuration, and the ability to use datasheets to correctly set up and verify hardware functionality.